

Deep Learning 1 — Übungsklausur 8

Technische Universität Berlin — Deep Learning 1

Gesamtpunkte: 100 | Bearbeitungszeit: 90 Minuten | Alle Hilfsmittel zugelassen

1 Multiple Choice (20 Punkte)

8 Fragen à 2,5 Punkte. Nur eine Antwort ist korrekt.

1. Was ist der Unterschied zwischen Cross-Entropy Loss und Log-Loss für binäre Klassifikation?

- (a) Cross-Entropy ist für Regression, Log-Loss für Klassifikation
- (b) Sie sind äquivalent — Log-Loss ist Cross-Entropy für binären Fall
- (c) Cross-Entropy minimiert Perplexity, Log-Loss die Genauigkeit
- (d) Log-Loss ist nur für SVM definiert

2. Welche Klassen von RNN-Architekturen sind für Seq2Seq-Aufgaben geeignet?

- (a) Nur Standard (unidirektionale) RNNs
- (b) Standard, Bidirektionale und Encoder-Decoder RNNs
- (c) Nur Encoder-Decoder RNNs
- (d) Nur Bidirektionale RNNs

3. Wie viele Parameter hat `torch.nn.Conv2d(3, 20, 5)` inklusive Bias?

- (a) 1520
- (b) 1500
- (c) 320
- (d) 1505

4. Was ist Topological Sparse Coding?

- (a) Sparse Coding mit hierarchischen Strukturen
- (b) Sparse Coding mit der Einschränkung, dass Quelldimensionen auf einem 2D-Gitter korrelieren
- (c) Sparse Coding für CNN-Feature-Maps
- (d) Sparse Coding mit L2-Norm

5. Was ist die Ausgabe bei $y = \tanh(0)$?

- (a) 1
- (b) 0
- (c) 0.5
- (d) -1

6. Welche Aussage zum Encoder-Decoder RNN ist korrekt?

- (a) Der Encoder verarbeitet den Output, der Decoder den Input
- (b) Zuerst wird die Eingangssequenz zu einer Repräsentation s kodiert, dann wird daraus die Ausgangssequenz erzeugt
- (c) Encoder und Decoder teilen sich keine Parameter
- (d) Der Encoder-Decoder verarbeitet nur feste Sequenzlängen

7. Was versteht man unter dem 'Texture Bias' Problem bei CNNs?

- (a) CNNs können keine Texturen lernen
- (b) CNNs lernen bevorzugt Texturfeatures statt Formfeatures, wodurch leichte Texturveränderungen die Vorhersage stark ändern können

- (c) CNNs sind gegenüber Texturänderungen invariant
- (d) Das ist kein reales Problem

8. Was beschreibt Dataset Bias und wie kann es das Modell beeinflussen?

- (a) Bias in den Modellgewichten durch falsche Initialisierung
- (b) Überrepräsentation bestimmter Verteilungsbereiche, die das Modell zu falschen Annahmen über die Gesamtverteilung führt
- (c) Ein zu kleines Trainingsset
- (d) Numerische Instabilität bei der Berechnung

2 Theorie (20 Punkte)

Betrachten Sie das Lernproblem in Deep Learning:

(a)

Formulieren Sie das allgemeine Lernproblem in Deep Learning: Schreiben Sie den Trainings-Loss auf. Was ist der Unterschied zwischen Trainings- und True-Error? (8 Pkt.)

Antwort:

(b)

Erklären Sie den Begriff 'endliche Daten' und welche Probleme er mit sich bringt (Overfitting, Spurious Correlations). (6 Pkt.)

Antwort:

(c)

Was bedeutet 'Best-Complexity' nach Vapnik? Wie wählt man optimale Modellkomplexität? (6 Pkt.)

Antwort:

3 Theorie (20 Punkte)

Betrachten Sie verschiedene Optimierungsansätze für das Autoencoder-Problem:

(a)

Schreiben Sie das Optimierungsziel für einen einfachen linearen Autoencoder (ohne Aktivierungsfunktion) auf. Was ist der Zusammenhang zu PCA? (8 Pkt.)

Antwort:

(b)

Warum reicht eine lineare Encoder-Funktion nicht aus? Wie löst man dieses Problem? (6 Pkt.)

Antwort:

(c)

Was ist topological sparse coding? Beschreiben Sie die zusätzliche Einschränkung und warum sie sinnvoll sein kann. (6 Pkt.)

Antwort:

4 Theorie (20 Punkte)

Betrachten Sie Shapley Values und LRP im Vergleich:

(a)

Schreiben Sie die Formel der Shapley Values auf und erklären Sie alle Terme. (8 Pkt.)

Antwort:

(b)

Was ist das computationale Hauptproblem bei exakten Shapley Values und wie wird es gelöst? (6 Pkt.)

Antwort:

(c)

Vergleichen Sie LRP und Shapley Values: In welchen Situationen bevorzugen Sie welche Methode? (6 Pkt.)

Antwort:

5 Programmierung (20 Punkte)

Regularisierung implementieren:

(a)

Zeigen Sie, wie Weight Decay in PyTorch implementiert wird (über den Optimizer). Erklären Sie den Zusammenhang zwischen `weight_decay`-Parameter und dem L2-Regularisierungsterm im Loss. (8 Pkt.)

Antwort:

(b)

Implementieren Sie einen Dropout-Layer manuell (ohne `nn.Dropout`) in PyTorch. Der Layer soll Bernoulli(p)-Rauschen auf die Aktivierungen anwenden, nur beim Training. (12 Pkt.)

Antwort:
